

Proyecto: Intelligent AI Model Routing Platform

1. Descripción general

La empresa utiliza modelos de inteligencia artificial para responder consultas de atención al cliente, resumir documentos, extraer información y analizar solicitudes.

Actualmente, todas las consultas son enviadas al mismo modelo de IA, independientemente de su complejidad. Esto genera varios problemas:

- Costos innecesariamente altos.
- Mayor tiempo de respuesta.
- Dependencia de un único proveedor.
- Falta de control sobre la calidad de las respuestas.
- Dificultad para detectar errores o degradaciones.
- Falta de información sobre qué modelo funciona mejor para cada caso.

Se requiere desarrollar una plataforma intermediaria que reciba las solicitudes de las aplicaciones empresariales y seleccione automáticamente el modelo de IA más adecuado.

El sistema deberá elegir el modelo más económico que pueda resolver cada solicitud manteniendo un nivel mínimo de calidad, latencia y confiabilidad.

2. Objetivo del proyecto

Construir una plataforma que permita:

- Utilizar diferentes modelos y proveedores de IA.
- Seleccionar automáticamente el modelo más adecuado para cada solicitud.
- Reducir costos sin afectar significativamente la calidad.
- Escalar hacia modelos más potentes cuando una respuesta no sea satisfactoria.
- Cambiar de proveedor si uno falla.
- Medir calidad, costo y latencia.
- Mantener trazabilidad completa de cada solicitud.
- Configurar políticas diferentes para cada cliente o aplicación.

La plataforma no pretende ser un chatbot final.

Funcionará como infraestructura para que otras aplicaciones puedan utilizar modelos de IA de forma confiable, segura y eficiente.

3. Caso de uso principal

Una empresa posee una plataforma de atención al cliente.

Los usuarios pueden enviar consultas como:

- "No puedo cambiar mi contraseña."
- "Me cobraron dos veces la misma factura."
- "Quiero comparar estos dos movimientos bancarios."
- "Analizá el contrato adjunto y decime si puedo cancelarlo sin multa."

Todas estas solicitudes pertenecen al área de atención al cliente, pero presentan diferentes niveles de dificultad.

La plataforma deberá analizar las características de cada solicitud y seleccionar automáticamente un modelo.

Ejemplo:

Solicitud	Modelo seleccionado
Pregunta sencilla y corta	Modelo económico
Reclamo con información financiera	Modelo intermedio
Análisis de contrato extenso	Modelo avanzado
Solicitud con información sensible	Modelo local o autorizado
Proveedor principal caído	Proveedor alternativo

4. Usuarios del sistema

4.1 Aplicación cliente

Es el sistema empresarial que consume la API.

Por ejemplo:

- Plataforma de atención al cliente.
- Sistema de análisis documental.
- Aplicación de recursos humanos.
- Sistema de clasificación de correos.
- Plataforma de procesamiento de facturas.

4.2 Administrador

Es la persona encargada de configurar:

- Proveedores.
- Modelos disponibles.
- Límites de costo.
- Requisitos mínimos de calidad.
- Políticas de enrutamiento.
- Clientes y credenciales.
- Casos de evaluación.

4.3 Analista

Es la persona que consulta:

- Costos.
- Latencia.
- Calidad.
- Errores.
- Uso por cliente.
- Comparación entre modelos.
- Solicitudes escaladas.
- Fallos de proveedores.

5. Alcance de la primera versión

La primera versión deberá permitir:

1. Registrar aplicaciones clientes.
2. Autenticar solicitudes mediante API keys.
3. Registrar diferentes modelos de IA.
4. Conectarse inicialmente con dos modelos.
5. Recibir solicitudes mediante una API común.
6. Extraer características básicas de cada solicitud.
7. Filtrar modelos que no cumplen los requisitos.
8. Seleccionar automáticamente un modelo.
9. Ejecutar la solicitud.
10. Validar la respuesta.
11. Escalar a otro modelo si la respuesta falla.
12. Utilizar un proveedor alternativo si el primero no responde.
13. Registrar costo, latencia y resultado.
14. Consultar el historial de solicitudes.
15. Mostrar métricas básicas en un panel administrativo.

6. Fuera del alcance inicial

La primera versión no incluirá:

- Entrenamiento de modelos propios.
- Fine-tuning.
- Un sistema completo de atención al cliente.
- Facturación real a clientes.
- Kubernetes.
- Microservicios completos.
- Modelos de aprendizaje automático para el routing.
- Procesamiento de audio o video.
- Alta disponibilidad entre múltiples regiones.
- Integración con más de dos proveedores.

Estas funcionalidades podrán incorporarse posteriormente.

7. Flujo principal

7.1 Recepción de la solicitud

La aplicación cliente envía una solicitud a la plataforma.

Ejemplo:

```
{
  "task_type": "customer_support",
  "input": "Me cobraron dos veces la factura 18392.",
  "language": "es",
  "priority": "normal",
  "max_cost_usd": 0.05,
  "max_latency_ms": 5000,
  "minimum_quality": 0.85,
  "contains_sensitive_data": true,
  "output_format": "json"
}
```

7.2 Validación inicial

La plataforma deberá comprobar:

- Que la API key sea válida.
- Que el cliente esté activo.
- Que los campos obligatorios estén presentes.
- Que el cliente no haya superado su límite.
- Que el presupuesto sea válido.
- Que el tamaño del contenido esté permitido.

7.3 Extracción de características

El sistema deberá obtener características objetivas de la solicitud.

Entre ellas:

- Cantidad estimada de tokens.
- Longitud del texto.
- Idioma.
- Existencia de archivos.
- Tipo de archivo.
- Necesidad de visión.
- Formato de salida requerido.
- Nivel de prioridad.
- Información sensible.
- Presupuesto máximo.
- Latencia máxima.
- Calidad mínima requerida.

Inicialmente, estas características serán obtenidas mediante reglas deterministas y metadatos enviados por la aplicación cliente.

7.4 Obtención de modelos candidatos

El sistema deberá descartar modelos que no cumplan las condiciones.

Por ejemplo:

- Modelo que no soporta imágenes.
- Modelo cuyo costo supera el máximo permitido.
- Modelo cuya latencia estimada supera el límite.
- Modelo no habilitado para información sensible.
- Modelo que no admite la cantidad de tokens.
- Modelo no disponible.
- Modelo no autorizado para ese cliente.

7.5 Selección del modelo

Entre los candidatos restantes, el sistema calculará una puntuación.

Ejemplo conceptual:

```
score =  
    calidad esperada × peso_calidad  
    - costo estimado × peso_costo  
    - latencia estimada × peso_latencia  
    + confiabilidad × peso_confiabilidad
```

El modelo con mayor puntuación será seleccionado.

La calidad esperada deberá obtenerse de evaluaciones previas almacenadas en la plataforma.

7.6 Ejecución

La plataforma deberá transformar la solicitud común al formato requerido por el proveedor seleccionado.

Después deberá:

- Enviar la solicitud.
- Medir la latencia.
- Calcular el consumo de tokens.
- Estimar el costo.
- Guardar el estado de la operación.

7.7 Validación de la respuesta

La plataforma deberá comprobar:

- Que exista una respuesta.
- Que el formato sea válido.
- Que el JSON pueda ser procesado.
- Que estén presentes los campos requeridos.
- Que la respuesta no esté vacía.
- Que no haya sido bloqueada por el proveedor.
- Que se hayan cumplido reglas de negocio específicas.

7.8 Escalado

Si la respuesta no supera las validaciones, el sistema deberá intentar con un modelo más potente.

Flujo esperado:

```
Modelo económico
    ↓
Validación
    ↓ falla
Modelo intermedio
    ↓
Validación
    ↓ falla
Modelo avanzado
    ↓
Revisión humana o error controlado
```

7.9 Respuesta final

La plataforma devolverá a la aplicación cliente:

```
{
  "request_id": "req_90123",
  "status": "completed",
  "output": {
    "category": "duplicate_charge",
    "priority": "high",
    "suggested_response": "Revisaremos el cobro duplicado."
  },
  "routing": {
    "selected_model": "model-medium",
    "provider": "provider-a",
    "attempts": 2,
    "fallback_used": true
  },
  "usage": {
    "input_tokens": 72,
    "output_tokens": 96,
    "latency_ms": 1320,
    "estimated_cost_usd": 0.006
  }
}
```

8. Modelos disponibles

Cada modelo deberá registrarse con la siguiente información:

- Identificador.
- Nombre.
- Proveedor.
- Versión.
- Estado.
- Costo de entrada.
- Costo de salida.
- Latencia promedio.
- Límite de contexto.
- Soporte de imágenes.
- Soporte de documentos.
- Soporte de salida estructurada.
- Idiomas soportados.
- Nivel de privacidad.
- Calidad obtenida por tipo de tarea.
- Porcentaje de disponibilidad.
- Cantidad máxima de solicitudes por minuto.

Ejemplo:

```
{
  "id": "model-small",
  "provider": "provider-a",
  "enabled": true,
  "input_cost_per_million_tokens": 0.20,
  "output_cost_per_million_tokens": 0.80,
  "average_latency_ms": 650,
  "context_window": 128000,
  "supports_vision": false,
  "supports_structured_output": true,
  "sensitive_data_allowed": false
}
```

9. Políticas de routing

Cada cliente podrá tener su propia política.

Ejemplo:

```
customer_support:
  minimum_quality: 0.85
  maximum_cost_usd: 0.05
  maximum_latency_ms: 5000
  routing_strategy: balanced
  allow_fallback: true
  maximum_attempts: 3

legal_analysis:
  minimum_quality: 0.95
  maximum_cost_usd: 0.50
  routing_strategy: quality_first
  require_human_review: true
```

Las estrategias iniciales serán:

Cheapest

Selecciona el modelo más económico que alcance la calidad mínima.

Fastest

Selecciona el modelo con menor latencia estimada.

Quality first

Prioriza calidad sobre costo y velocidad.

Balanced

Busca un equilibrio entre calidad, costo y latencia.

10. Evaluación de modelos

La plataforma deberá permitir crear datasets de evaluación.

Cada caso contendrá:

- Tipo de tarea.
- Entrada.
- Resultado esperado.
- Reglas de validación.

- Nivel de dificultad.
- Idioma.
- Etiquetas adicionales.

Ejemplo:

```
{
  "task_type": "customer_support",
  "input": "Me cobraron dos veces la misma factura.",
  "expected_output": {
    "category": "duplicate_charge",
    "priority": "high"
  },
  "difficulty": "medium",
  "language": "es"
}
```

El sistema ejecutará cada caso con los modelos disponibles y registrará:

- Resultado obtenido.
- Corrección.
- Latencia.
- Tokens utilizados.
- Costo.
- Errores.
- Validez del formato.

Posteriormente calculará métricas como:

- Exactitud.
- Porcentaje de JSON válido.
- Costo promedio.
- Latencia promedio.
- Tasa de error.
- Calidad por idioma.
- Calidad por dificultad.
- Calidad por tipo de tarea.

Estas métricas serán utilizadas por el router.

11. Aprendizaje basado en resultados

En la primera versión, el sistema no entrenará un modelo propio.

Sin embargo, deberá actualizar las estadísticas utilizando los resultados de evaluaciones y solicitudes reales revisadas.

Ejemplo:

Modelo pequeño
Consultas simples en español: 93% de calidad
Consultas complejas en español: 64%
Consultas simples en portugués: 72%

A partir de estos datos, el router podrá dejar de utilizar ese modelo para solicitudes complejas o en portugués.

La decisión continuará siendo determinista, pero estará basada en información histórica actualizada.

12. Fallback entre proveedores

Si un proveedor devuelve un error, supera el tiempo máximo o no está disponible, la plataforma deberá intentar con otro proveedor compatible.

Errores que activarán fallback:

- Timeout.
- Error HTTP 429.
- Error HTTP 500.
- Proveedor temporalmente no disponible.
- Respuesta vacía.
- Respuesta inválida.
- Límite de uso alcanzado.

El sistema deberá evitar reintentos infinitos.

Cada solicitud tendrá:

- Cantidad máxima de intentos.
- Tiempo máximo total.
- Lista de modelos ya utilizados.
- Motivo de cada fallo.

13. Circuit breaker

Si un proveedor falla repetidamente, el sistema deberá dejar de enviarle solicitudes temporalmente.

Estados:

- Closed: funcionamiento normal.
- Open: proveedor bloqueado temporalmente.
- Half-open: se permiten algunas solicitudes de prueba.

Ejemplo:

Si un proveedor falla 5 veces en 1 minuto:
abrir circuito durante 30 segundos.

14. Rate limiting

La plataforma deberá limitar la cantidad de solicitudes por cliente.

Ejemplos:

- 100 solicitudes por minuto.
- 10 solicitudes simultáneas.
- USD 20 de consumo diario.
- 1 millón de tokens mensuales.

Cuando un límite sea superado, la plataforma devolverá un error controlado.

15. Idempotencia

La API deberá aceptar una clave de idempotencia.

Ejemplo:

Idempotency-Key: operation-18273

Si la aplicación repite accidentalmente la misma solicitud, la plataforma no deberá ejecutarla dos veces.

Deberá devolver el resultado previamente almacenado.

16. Seguridad

La plataforma deberá incluir:

- Autenticación mediante API keys.
- Almacenamiento seguro de credenciales.
- Hash de API keys.
- Cifrado de secretos.
- Control de permisos.
- Enmascaramiento de datos sensibles.
- Registro de auditoría.
- Protección contra solicitudes excesivamente grandes.
- Validación estricta de inputs.
- No registrar secretos ni información sensible en logs.

No deberán guardarse las API keys de los proveedores directamente en el código fuente.

17. Datos sensibles

La solicitud podrá indicar que contiene información sensible.

```
{  
  "contains_sensitive_data": true  
}
```

Cuando este campo sea verdadero, la plataforma deberá:

- Utilizar solamente modelos autorizados.
- Evitar guardar el contenido completo cuando la política lo prohíba.
- Enmascarar datos personales en logs.
- Registrar quién realizó la solicitud.
- Aplicar políticas especiales de retención.

En una versión posterior, la plataforma podrá detectar automáticamente:

- Documentos de identidad.
 - Números de tarjeta.
 - Correos.
 - Teléfonos.
 - Direcciones.
 - Información médica.
-

18. Observabilidad

Cada solicitud deberá contar con un identificador único.

El sistema deberá registrar:

- Cliente.
- Tipo de tarea.
- Modelo seleccionado.
- Proveedor.
- Cantidad de intentos.
- Duración.
- Tokens.
- Costo.
- Estado.
- Errores.
- Fallbacks.
- Validaciones fallidas.

Las métricas principales serán:

- Solicitudes por minuto.
- Latencia promedio.
- Latencia p95 y p99.
- Costo por cliente.
- Costo por tarea.
- Uso por modelo.
- Uso por proveedor.
- Tasa de errores.
- Tasa de fallback.
- Porcentaje de respuestas válidas.
- Disponibilidad de proveedores.

19. Panel administrativo

El panel deberá mostrar:

Dashboard general

- Solicitudes totales.
- Solicitudes exitosas.
- Solicitudes fallidas.
- Costo total.
- Latencia promedio.

- Modelo más utilizado.
- Proveedor con más errores.

Modelos

- Lista de modelos.
- Estado.
- Costos.
- Latencia.
- Calidad.
- Disponibilidad.

Solicitudes

- Historial.
- Modelo seleccionado.
- Intentos.
- Costos.
- Errores.
- Respuesta obtenida.

Evaluaciones

- Datasets disponibles.
- Resultados por modelo.
- Comparación de calidad.
- Comparación de costos.
- Regresiones detectadas.

Clientes

- Estado.
- Límites.
- Uso.
- Presupuesto.
- API keys.
- Políticas habilitadas.

20. API inicial

Crear una solicitud

```
POST /api/v1/inference
```

Consultar una solicitud

```
GET /api/v1/requests/{request_id}
```

Listar solicitudes

```
GET /api/v1/requests
```

Registrar un modelo

```
POST /api/v1/models
```

Listar modelos

```
GET /api/v1/models
```

Modificar un modelo

```
PATCH /api/v1/models/{model_id}
```

Crear una política

```
POST /api/v1/routing-policies
```

Ejecutar una evaluación

```
POST /api/v1/evaluations/run
```

Consultar resultados

```
GET /api/v1/evaluations/{evaluation_id}
```

21. Modelo de datos inicial

Client

- id
- name
- status
- created_at

ApiKey

- id
- client_id
- key_hash
- created_at
- expires_at
- revoked_at

ModelProvider

- id
- name
- status
- configuration

AIModel

- id
- provider_id
- name
- version
- enabled
- input_cost
- output_cost
- average_latency
- context_window
- capabilities
- privacy_level

RoutingPolicy

- id
- client_id
- task_type
- strategy
- minimum_quality

- maximum_cost
- maximum_latency
- maximum_attempts
- allow_fallback

InferenceRequest

- id
- client_id
- task_type
- input
- status
- requested_at
- completed_at
- maximum_cost
- minimum_quality
- maximum_latency
- idempotency_key

InferenceAttempt

- id
- request_id
- model_id
- attempt_number
- status
- latency
- input_tokens
- output_tokens
- cost
- error_type
- error_message

EvaluationDataset

- id
- name
- task_type
- version
- created_at

EvaluationCase

- id
- dataset_id
- input
- expected_output

- difficulty
- language

EvaluationResult

- id
- case_id
- model_id
- score
- latency
- cost
- valid_output
- executed_at

22. Arquitectura inicial

La primera versión deberá implementarse como un monolito modular.

Módulos sugeridos:

```
authentication
clients
models
providers
routing
inference
validation
evaluations
billing
observability
audit
```

Flujo:

```
Aplicación cliente
↓
API Gateway
↓
Autenticación
↓
Extracción de características
↓
Motor de routing
```

```

    ↓
Adaptador del proveedor
    ↓
Validación
    ↓
Persistencia y métricas
    ↓
Respuesta

```

No se deberán implementar microservicios hasta que exista una necesidad real de escalado o aislamiento.

23. Stack tecnológico recomendado

Backend

- Python 3.12.
- FastAPI.
- Pydantic.
- SQLAlchemy.
- Alembic.

Base de datos

- PostgreSQL.

Cache y rate limiting

- Redis.

Procesamiento en segundo plano

- Celery, Dramatiq o una cola sencilla basada en Redis.

Observabilidad

- OpenTelemetry.
- Prometheus.
- Grafana.

Frontend

- React.
- TypeScript.
- Una librería simple de componentes.

Infraestructura

- Docker.
- Docker Compose.
- GitHub Actions.

Testing

- Pytest.
- Testcontainers.
- Locust o k6 para pruebas de carga.

24. Requisitos no funcionales

Rendimiento

- El routing interno deberá tardar menos de 100 milisegundos.
- La plataforma deberá soportar inicialmente 50 solicitudes concurrentes.
- Las operaciones administrativas no deberán bloquear las inferencias.

Disponibilidad

- Los errores de un proveedor no deberán detener toda la plataforma.
- La aplicación deberá continuar funcionando mientras exista al menos un modelo compatible disponible.

Mantenibilidad

- Cada proveedor deberá implementarse mediante un adaptador común.
- Agregar un proveedor no deberá modificar el motor principal.
- La lógica de routing deberá estar separada de la integración con proveedores.

Trazabilidad

- Cada decisión del router deberá ser explicable.
- El sistema deberá registrar por qué seleccionó un modelo.

Ejemplo:

```
{
  "selected_model": "model-medium",
  "reason": {
    "required_quality": 0.85,
    "expected_quality": 0.91,
```

```
"estimated_cost": 0.006,  
"estimated_latency_ms": 1200,  
"discarded_models": {  
  "model-small": "quality_below_minimum",  
  "model-large": "higher_cost_without_significant_quality_gain"  
}  
}  
}
```

25. Criterios de aceptación del MVP

El MVP se considerará completo cuando:

1. Una aplicación pueda autenticarse mediante API key.
2. Se puedan configurar al menos dos modelos.
3. Ambos modelos utilicen una interfaz común.
4. El sistema pueda recibir una solicitud.
5. El router pueda seleccionar un modelo utilizando costo, latencia y calidad.
6. El sistema pueda ejecutar la solicitud.
7. Se registre el costo estimado.
8. Se registre la latencia.
9. Se registre la cantidad de tokens.
10. Una respuesta inválida active el fallback.
11. Un error del proveedor active otro modelo.
12. Una clave de idempotencia evite ejecuciones duplicadas.
13. El historial pueda consultarse mediante API.
14. Exista un dataset de al menos 50 casos.
15. Los modelos puedan compararse mediante evaluaciones.
16. Existan pruebas unitarias y de integración.
17. El proyecto pueda levantarse mediante Docker Compose.
18. La documentación permita ejecutarlo localmente.

26. Primera demo esperada

La demo deberá utilizar una tarea de clasificación de consultas de atención al cliente.

Categorías:

- password_problem
- duplicate_charge
- refund_problem
- account_blocked

- general_question

La plataforma contará con:

- Un modelo económico.
- Un modelo más potente.
- Un dataset de 50 a 100 consultas.
- Una calidad mínima configurable.

La demo deberá mostrar:

1. Una consulta simple enviada al modelo económico.
 2. Una consulta compleja enviada directamente al modelo potente.
 3. Una respuesta inválida del modelo económico.
 4. El escalado automático al modelo potente.
 5. La caída simulada de un proveedor.
 6. El uso automático del proveedor alternativo.
 7. La diferencia de costo entre utilizar routing y utilizar siempre el modelo potente.
-

27. Fases de desarrollo

Fase 1: Base del backend

- Crear proyecto FastAPI.
- Configurar PostgreSQL.
- Crear migraciones.
- Crear clientes.
- Crear API keys.
- Registrar modelos.
- Crear estructura de proveedores.

Fase 2: Inferencia común

- Crear endpoint de inferencia.
- Implementar adaptador del primer proveedor.
- Implementar adaptador del segundo proveedor.
- Normalizar respuestas.
- Guardar solicitudes e intentos.

Fase 3: Motor de routing

- Calcular costo estimado.
- Filtrar modelos incompatibles.
- Implementar estrategias.
- Explicar cada decisión.

- Incorporar políticas por cliente.

Fase 4: Validación y fallback

- Validar JSON.
- Validar esquemas.
- Implementar reintentos.
- Implementar fallback.
- Implementar circuit breaker.
- Implementar idempotencia.

Fase 5: Evaluaciones

- Crear datasets.
- Ejecutar casos con diferentes modelos.
- Calcular métricas.
- Guardar calidad por modelo.
- Utilizar resultados en el routing.

Fase 6: Observabilidad

- Logs estructurados.
- Métricas.
- Trazas.
- Panel de Grafana.
- Alertas básicas.

Fase 7: Panel administrativo

- Dashboard.
- Historial.
- Modelos.
- Evaluaciones.
- Clientes.
- Políticas.

Fase 8: Calidad final

- Pruebas unitarias.
- Pruebas de integración.
- Pruebas de carga.
- Documentación.
- Diagrama de arquitectura.
- Video de demostración.
- Deployment público.

28. Primera tarea de desarrollo

Historia de usuario

Como aplicación cliente, quiero enviar una consulta a través de una API común para recibir una respuesta de un modelo de IA sin conocer directamente qué proveedor será utilizado.

Criterios de aceptación

- Existe el endpoint `POST /api/v1/inference`.
- La solicitud se valida con Pydantic.
- Se verifica la API key.
- Se crea un registro en la base de datos.
- Se selecciona inicialmente un modelo configurado como predeterminado.
- La solicitud es enviada al proveedor.
- Se almacena la latencia.
- Se almacenan los tokens.
- Se almacena el costo estimado.
- Se devuelve un identificador único.
- Los errores del proveedor son controlados.

29. Nombre sugerido para el repositorio

```
intelligent-model-router
```

Otras opciones:

```
ai-routing-platform  
model-gateway  
smart-llm-router  
ai-inference-gateway
```

30. Resultado esperado del portfolio

El proyecto deberá demostrar experiencia en:

- Backend.
- Diseño de APIs.
- Integración de inteligencia artificial.

- Sistemas distribuidos.
- Evaluación de modelos.
- Optimización de costos.
- Seguridad.
- Resiliencia.
- Observabilidad.
- Arquitectura cloud.
- Testing.
- Documentación técnica.

Descripción corta para el portfolio:

Built an AI inference gateway that automatically selects the most cost-effective language model based on quality, latency, privacy and budget constraints. The platform supports model fallback, structured-output validation, provider failover, evaluation datasets, cost tracking and complete request observability.